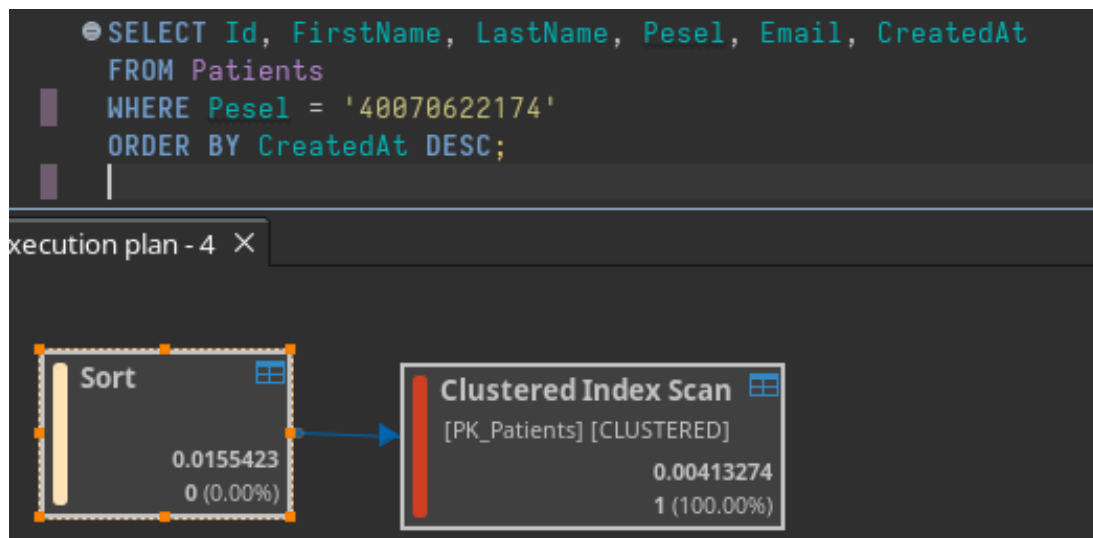


Przypadek 1: Wyszukiwanie Pacjenta po PESEL bez dedykowanego indeksu



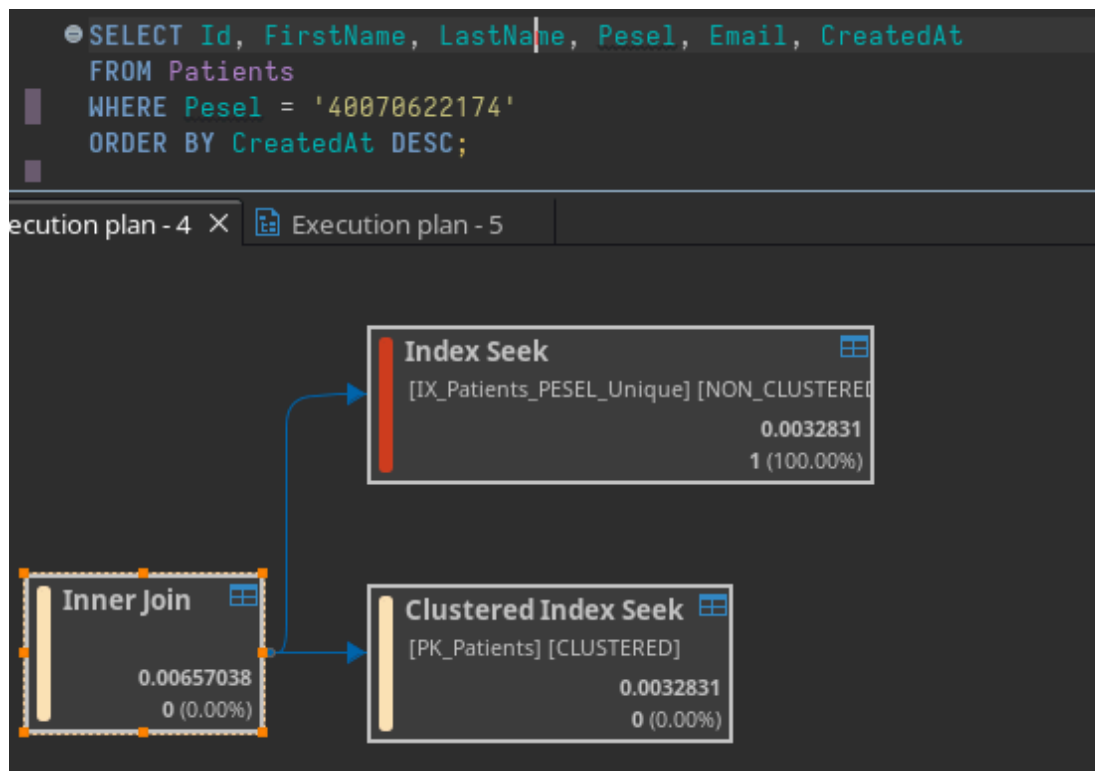
Rysunek 1: Plan Wykonania (Clustered Index Scan)

Opis działania i analiza wydajności

W tym scenariuszu system wykonuje wyszukiwanie konkretnego pacjenta na podstawie unikalnego numeru PESEL. Z powodu braku indeksu nieklastrowanego na kolumnie Pesel, optymalizator zapytań SQL Server nie posiada żadnego punktu odniesienia.

1. **Clustered Index Scan:** Silnik bazy danych jest zmuszony do fizycznego przeszukania całej struktury tabeli Patients (która domyślnie jest indeksem klastrowanym PK_Patients) wiersz po wierszu. Przy dużej skali bazy generuje to ogromny narzut operacji I/O.
2. **Kosztowna operacja Sort**

Przypadek 2: Wyszukiwanie Pacjenta z wykorzystaniem indeksu nieklastrowanego



Rysunek 2: Plan Wykonania (Index Seek + Key Lookup)

Dodanie indeksu

W celu optymalizacji tego zapytania, w strukturze tabeli Patients zaimplementowano poniższy indeks unikalny:

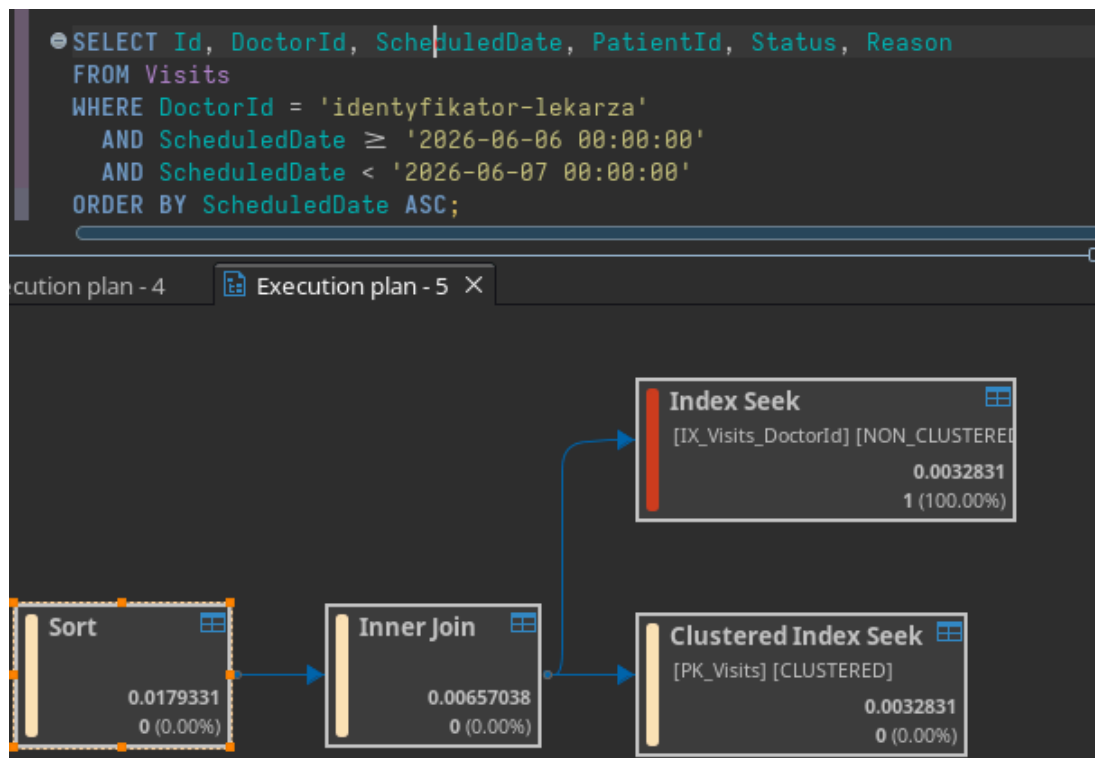
```
modelBuilder.Entity<Patient>()  
    .HasIndex(p => p.Pesel)  
    .IsUnique()  
    .HasDatabaseName("IX_Patients_PESSEL_Unique");
```

Opis działania i analiza wydajności

Po dodaniu indeksu nieklastrowanego IX_Patients_PESSEL_Unique, sytuacja mocno się zmienia - zniknął kosztowny krok sortowania.

1. **Index Seek:** Silnik wykonuje bezpośredni, precyzyjny skok do drzewa B-Tree indeksu na kolumnie Pesel, natychmiast odnajdując właściwy rekord.
2. **Clustered Index Seek / Key Lookup (Dolny węzeł):** Ponieważ indeks zawiera jedynie kolumnę Pesel oraz referencję do klucza głównego, silnik musi dociągnąć pozostałe kolumny.

Przypadek 3: Pobieranie harmonogramu wizyt lekarza na dany dzień



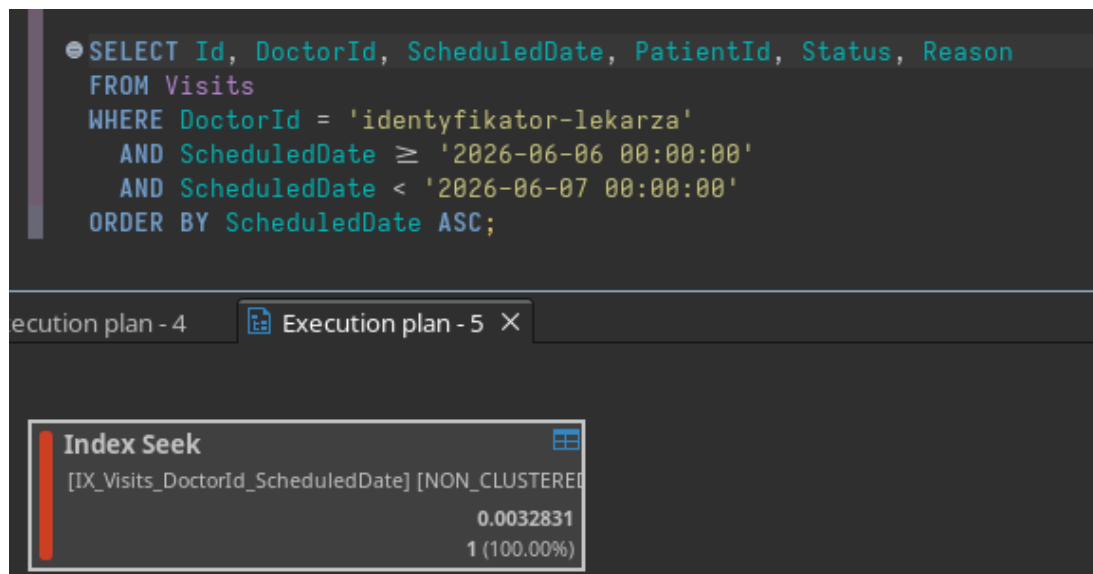
Rysunek 3: Plan Wykonania (Index Seek + Sort)

Opis działania i analiza wydajności

Zapytanie filtruje tabelę Visits na podstawie identyfikatora lekarza (DoctorId) oraz przedziału czasowego (bieżący dzień).

1. **Index Seek:** Optymalizator poprawnie wykorzystuje indeks IX_Visits_DoctorId. Pozwala to na szybkie odfiltrowanie wizyt przypisanych do wybranego medyka.
2. **Inner Join (Key Lookup):** Podobnie jak w przypadku drugim, indeks nie zawiera wszystkich wymaganych przez klauzulę SELECT pól (np. Status, Reason), co powoduje konieczność sięgnięcia do tabeli głównej PK_Visits.
3. **Operacja Sort:** Wyniki są jawnie sortowane po ScheduledDate ASC. Indeks nie wspierał natywnego sortowania dla tego zestawu kolumn, co wymagało dodatkowego narzutu CPU.

Przypadek 4: Optymalny Stan Docelowy dla Harmonogramu Wizyt



Rysunek 4: Plan Wykonania (Index Seek)

Dodanie indeksu

W celu uzyskania powyższego optymalnego planu zapytania, w strukturze tabeli Visits zaimplementowano kompozytowy indeks pokrywający:

```
modelBuilder.Entity<Visit>()
    .HasIndex(v => new { v.DoctorId, v.ScheduledDate })
    .HasDatabaseName("IX_Visits_DoctorId_ScheduledDate")
    .IncludeProperties(v => new { v.PatientId, v.Status, v.Reason });
```

Opis działania i analiza wydajności

Wprowadzenie dedykowanego indeksu kompozytowego rozwiązuje jednocześnie dwa problemy wydajnościowe z Przypadku 3.

1. **Eliminacja operacji Sort:** Indeks został założony na dwie kolumny jednocześnie: DoctorId oraz ScheduledDate. Dzięki temu dane na dysku są już fizycznie ułożone w kolejności chronologicznej dla każdego lekarza. SQL Server nie musi tracić czasu na sortowanie - po prostu czyta rekordy z indeksu jeden po drugim.
2. **Eliminacja operacji Inner Join (Key Lookup):** Poprzez dołączenie pozostałych kolumn do struktury indeksu, zapytanie stało się w 100% „pokryte”. Cały plan wykonania sprowadza się do jednej, płaskiej i ultraszybkiej operacji **Index Seek**